UNIT 5 · TOPIC 5.6

Detecting Attacks on Data and Applications

Assignment, Answer Key

Teacher answer key, 30 points total.

Practice Questions

5.6.a · Application monitoring and logging

1. A single failed login for one user is unremarkable. Fifty failed logins from one IP across twenty usernames in two minutes is a signal. What does this illustrate? (1 pt)

- **● A. The pattern, not the individual event, is the signal**
- ○ B. Individual events matter more than patterns
- ○ C. Logs should never be reviewed manually
- ○ D. HTTP status codes are irrelevant to security

Why: Seen once, a failed login is unremarkable; seen repeatedly in a tight pattern from one source, it becomes a brute-force signal.

2. Select ALL fields a well-designed security log line typically includes. (3 pts)

- **■ A. Timestamp**
- **■ B. Requesting IP address**
- **■ C. Result/status code**
- □ D. The victim's home address

Why: Standard log fields are timestamp, source IP, user identity, resource accessed, and result code, never unrelated personal data like a home address.

3. What is a SIEM? (1 pt)

- ○ A. A type of firewall rule
- ○ B. An encryption algorithm
- ○ C. A programming language for writing exploits
- **● D. A platform that centralizes logs from many sources and applies detection rules to raise alerts**

Why: SIEM = Security Information and Event Management, a centralized log-aggregation and alerting platform.

4. Name the log severity level at which security-relevant events (like authentication attempts) should be recorded so they are not silently dropped in production. (1 pt)

Acceptable answers: INFO; info level; info or higher

Why: Security-relevant events should be logged at INFO or higher, since DEBUG is typically disabled in production.

5. Why do defenders stream logs to a remote aggregator in real time rather than only storing them locally on the application server? (1 pt)

- ○ A. It makes the application run faster
- **● B. An attacker who compromises the server can delete local logs to cover their tracks; a remote copy preserves the forensic record**
- ○ C. Local logs are illegal to store
- ○ D. It has no security benefit

Why: Protecting the logs themselves matters: real-time remote streaming prevents an attacker from erasing the evidence of their own intrusion.

5.6.b · Data loss detection (DLP)

6. Which DLP detection strategy inspects the actual data leaving the organization for patterns like credit-card numbers or SSNs? (1 pt)

○ A. Volume/behavior-based detection

○ B. Endpoint detection and response

● C. Content-based detection

○ D. Least privilege

Why: Content-based detection scans the data itself for recognizable sensitive patterns.

7. Select ALL statements that correctly describe volume/behavior-based DLP detection. (3 pts)

■ A. It can catch exfiltration of encrypted or compressed data that has no recognizable content pattern

■ B. It relies on comparing an account's current activity against its normal baseline

□ C. It requires reading and understanding the literal content of every file transferred

■ D. A sudden large transfer to an unfamiliar destination is a classic signal it looks for

Why: Behavior-based detection watches how much/when/where data moves, not what it contains, which is exactly why it catches encrypted exfiltration content inspection would miss.

8. Why do effective DLP programs maintain an approved-destination allowlist? (1 pt)

○ A. To make transfers faster

○ B. It is required to encrypt data

● C. So legitimate large transfers (backups, sanctioned partners) don't drown out real alerts with false positives

○ D. It replaces the need for any volume threshold

Why: An approved-destination list lets DLP distinguish sanctioned large transfers from suspicious ones, reducing alert noise.

9. Name the DLP deployment point that runs on individual laptops/workstations and can block a USB copy of a flagged file, which network DLP alone would never see. (1 pt)

Acceptable answers: endpoint dlp; endpoint

Why: Endpoint DLP instruments the device itself, catching exfiltration paths (USB, local print) that never touch the network.

10. How does DLP differ from a purely forensic control like log review? (1 pt)

○ A. DLP cannot detect anything until after data has already left

○ B. DLP and log review are identical in function

○ C. DLP only works on encrypted traffic

● D. A well-integrated DLP system can act preventively in real time, blocking or quarantining a transfer before it completes

Why: Unlike a log reviewed after the fact, DLP can intervene during the attempt, blocking an email or quarantining an upload before it finishes.

5.6.c · Digital evidence analysis and attack classification

11. A single failed-login spike is observed. What is the analyst's correct next step before declaring it a brute-force attack? (1 pt)

- **● A. Check the underlying evidence, e.g. is it one source targeting many usernames, or many sources targeting one username?**
- ○ B. Immediately reset all passwords
- ○ C. Ignore it, since one indicator is never worth investigating
- ○ D. Delete the log entries

Why: An alert is a hypothesis, not a verdict; the analyst must confirm with evidence before concluding the attack type, since the same alert could indicate brute-force or credential stuffing.

12. Select ALL statements that correctly describe an indicator of compromise (IOC). (3 pts)

- **■ A. A single IOC is rarely conclusive proof of a specific attack type by itself**
- **■ B. Examples include a suspicious IP, a known-malware file hash, or a credential-harvesting URL**
- **■ C. Attack classification typically requires reasoning about multiple IOCs together**
- □ D. An IOC always proves the attacker's exact identity

Why: IOCs are individually weak signals; classification comes from weighing multiple IOCs together, and none of them reveal an attacker's identity by themselves.

13. Why does chain of custody matter for evidence even outside a courtroom? (1 pt)

- ○ A. It doesn't matter outside a courtroom
- ○ B. It is only relevant for network logs
- ○ C. It only applies to physical evidence
- **● D. An internal investigation's credibility also depends on being able to show evidence wasn't tampered with**

Why: Documented chain of custody (who collected/accessed evidence, hash verification) protects the credibility of internal incident reports, not just legal proceedings.

14. Name the widely-used framework that gives analysts a shared vocabulary for attack techniques so classifications are consistent across teams. (1 pt)

Acceptable answers: MITRE ATT&CK; ATT&CK; MITRE ATTACK

Why: MITRE ATT&CK provides a common, consistent vocabulary for classifying attack techniques across analysts and teams.

15. An evidence bundle contains a known-malware file hash AND a suspicious email attachment. Which best describes correct classification reasoning? (1 pt)

- ○ A. Pick whichever category is listed first alphabetically without looking at the evidence
- **● B. Score how many known indicators support each category and communicate which is best-supported, noting remaining uncertainty if any**
- ○ C. Always classify as the most severe possible category
- ○ D. Ignore one of the two indicators since only one can be correct

Why: Mature analysis weighs all evidence, reports the best-supported classification, and honestly communicates any remaining uncertainty rather than discarding conflicting evidence.

Coding Assignments (Python 3)

5.6.a · Application monitoring and logging

3 pts · auto-graded on the site

Coding problem, 5.6.a: Brute-force detector (SIEM alert rule)

Skill: Detect Attacks · **Maps to:** 5.6.A · **Language:** Python 3 (pytest)

Problem

Implement the SIEM alert rule described in the lesson: given a list of parsed application log entries, return every IP address whose failed-login (401) count exceeds a threshold within a rolling time window, a classic brute-force signal.

Function contract

Implement `solve(entries: list, threshold: int, window_seconds: int) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `entries`: `list[dict]`, each `{"ip": str, "ts": int, "status": int}` (`ts` = seconds since start of log, not necessarily sorted).
- `threshold`: `int`, minimum number of `401` events within the window to flag the IP.
- `window_seconds`: `int`, rolling window size, in seconds (inclusive on both ends).

Output

A **sorted, de-duplicated** `list[str]` of flagged IP addresses.

Examples

```
entries = [
  {"ip":"10.0.0.1","ts":0,"status":401},
  {"ip":"10.0.0.1","ts":10,"status":401},
  {"ip":"10.0.0.1","ts":20,"status":401},
]
solve(entries, threshold=3, window_seconds=60) -> ["10.0.0.1"]
```

```
entries = [{"ip":"10.0.0.2","ts":0,"status":200}]
solve(entries, threshold=1, window_seconds=60) -> []   # success responses never count
```

Constraints

- Deterministic, offline, standard library only.
- Only `status == 401` entries count toward the brute-force count; other codes are ignored.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.6.a. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from collections import defaultdict
```

```
from typing import Dict, List


def solve(entries: List[Dict[str, object]], threshold: int, window_seconds: int) -> List[str]:
    """Flag source IPs whose failed-login count reaches >= threshold within some rolling
    window of window_seconds, per the SIEM brute-force rule described in the lesson."""
    by_ip: Dict[str, List[int]] = defaultdict(list)
    for e in entries:
        if e.get("status") == 401:
            by_ip[e["ip"]].append(e["ts"])

    flagged = set()
    for ip, times in by_ip.items():
        times = sorted(times)
        for start in times:
            count = sum(1 for t in times if start <= t <= start + window_seconds)
            if count >= threshold:
                flagged.add(ip)
                break

    return sorted(flagged)
```

5.6.b · Data loss detection (DLP)

3 pts · auto-graded on the site

Coding problem, 5.6.b: DLP egress rule

Skill: Detect Attacks · **Maps to:** 5.6.B · **Language:** Python 3 (pytest)

Problem

Implement the volume/behavior-based DLP rule described in the lesson: given a list of outbound transfer events, flag the ones that exceed a size threshold AND go to a destination that is not on the organization's approved-destination list, a classic egress-monitoring rule.

Function contract

Implement `solve(transfers: list, approved_destinations: list, volume_threshold_mb: int) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `transfers`: `list[dict]`, each `{"id": str, "bytes_mb": int, "destination": str}`.
- `approved_destinations`: `list[str]` of known-good destination names.
- `volume_threshold_mb`: `int`, transfer must exceed this size to be eligible for flagging.

Output

Sorted `list[str]` of flagged transfer `id`s.

Examples

```
transfers = [{"id":"t1","bytes_mb":500,"destination":"unknown-cloud.example"}]
solve(transfers, ["corp-backup.internal"], volume_threshold_mb=100) -> ["t1"]

transfers = [{"id":"t2","bytes_mb":500,"destination":"corp-backup.internal"}]
solve(transfers, ["corp-backup.internal"], volume_threshold_mb=100) -> []   # approved destination
```

Constraints

- Deterministic, offline, standard library only.
- The size comparison is strict (`bytes_mb > volume_threshold_mb`).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.6.b. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import Dict, List


def solve(
    transfers: List[Dict[str, object]],
    approved_destinations: List[str],
    volume_threshold_mb: int,
) -> List[str]:
    """DLP egress rule: flag transfer `id`s that exceed volume_threshold_mb AND whose
    destination is NOT in approved_destinations. Returns ids sorted ascending (as given,
    since ids are unique strings, sorted lexicographically)."""
    approved = set(approved_destinations)
    flagged = [
        t["id"]
        for t in transfers
        if t["bytes_mb"] > volume_threshold_mb and t["destination"] not in approved
    ]
    return sorted(flagged)
```

5.6.c · Digital evidence analysis and attack classification

3 pts · auto-graded on the site

Coding problem, 5.6.c: Attack classifier (evidence -> category)

Skill: Detect Attacks, Collaborate · **Maps to:** 5.6.B, 5.6.C · **Language:** Python 3 (pytest)

Problem

Formalize the analyst reasoning process from the lesson: given a bundle of evidence indicators collected during an investigation, score each known attack category by how many of its indicators are present, and return the best-supported classification.

Function contract

Implement `solve(indicators: list) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`indicators`: `list[str]` of evidence tags (may contain duplicates).

Known indicator-to-category mapping

- `"brute-force"`: `multiple_401_responses`, `single_source_ip`, `many_usernames_targeted`, `credential_guessing_pattern`

- "sql-injection": sql_meta_characters_in_request, database_error_in_response, union_select_pattern, unexpected_query_result_size
- "malware": known_malware_file_hash, unexpected_process_spawned, outbound_c2_connection, persistence_mechanism_installed
- "phishing": suspicious_email_attachment, credential_harvesting_url, spoofed_sender_domain, user_reported_suspicious_email

Output

str, the category with the most matching indicators (as a set; duplicates don't add extra weight). Ties go to the alphabetically first tied category. "unknown" if nothing matches.

Examples

```
["multiple_401_responses", "single_source_ip"]  ->  "brute-force"
["server_restarted"]                             ->  "unknown"
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in tests.py (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 5.6.c. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import List

# Known indicators per attack category (case-insensitive exact match against evidence tags).
_CATEGORY_INDICATORS = {
    "brute-force": {
        "multiple_401_responses",
        "single_source_ip",
        "many_usernames_targeted",
        "credential_guessing_pattern",
    },
    "sql-injection": {
        "sql_meta_characters_in_request",
        "database_error_in_response",
        "union_select_pattern",
        "unexpected_query_result_size",
    },
    "malware": {
        "known_malware_file_hash",
        "unexpected_process_spawned",
        "outbound_c2_connection",
        "persistence_mechanism_installed",
    },
    "phishing": {
        "suspicious_email_attachment",
        "credential_harvesting_url",
        "spoofed_sender_domain",
        "user_reported_suspicious_email",
    },
}
```

```
def solve(indicators: List[str]) -> str:
    """Classify an evidence bundle by scoring each category's matching indicator count.

    Returns the category name with the highest score. Ties are broken by choosing the
    alphabetically first category among the tied leaders. If no indicator matches any
    known category, returns "unknown".
    """
    evidence = set(indicators)
    scores = {
        category: len(evidence & known_indicators)
        for category, known_indicators in _CATEGORY_INDICATORS.items()
    }
    best_score = max(scores.values())
    if best_score == 0:
        return "unknown"
    winners = sorted(cat for cat, score in scores.items() if score == best_score)
    return winners[0]
```